

# LUMINARY

## AI Destekli Sınav Hazırlık ve Eğitim Yönetim Platformu

Teknik Sistem Tasarımı, Tech Stack ve Ürün Yol Haritası

*Version: Project Planning 1*

Kaynak temel: Luminary Project Overview & Product Vision

Ek constraint: 2 kişilik ekip + düşük başlangıç sermayesi + GitHub merkezli geliştirme

## 1. Belgenin Amacı ve Kapsamı

Bu doküman, Luminary için daha önce çıkarılan sistem tasarımını teknik bir uygulama planına dönüştürür. Tasarımın amacı yalnızca “hangi teknoloji kullanılacak?” sorusunu yanıtlamak değildir; her teknolojinin hangi probleme hizmet ettiğini, bir feature için isteğin hangi katmanlardan geçtiğini, hangi verinin nerede tutulduğunu ve iki kişilik bir ekiple hangi bileşenlerin ilk günden kurulup hangilerinin ertelenmesi gerektiğini açıklamaktır.

Ürün vizyonunun çekirdeği, öğrencinin çalışma davranışından veri toplamak, bu veriyi anlamlandırmak, kişiselleştirilmiş aksiyon üretmek ve aynı veri ekosistemini kuruma görünür kılmaktır. Uzun vadede hedef “AI soru üreticisi” olmak değil, AI destekli Student Intelligence & Guidance Platform olmaktır. Bu nedenle mimarinin merkezinde öğrenci verisi ve bunun etrafındaki analitik/öneri döngüsü bulunur.

Bu rapor MVP’yi bilinçli şekilde sınırlar: ilk sürümde B2B kurum deneyimi, öğrenci tarafındaki çekirdek çalışma deneyimi ve ölçüm → analiz → öneri döngüsü güçlü hale getirilir. Gelişmiş sosyal özellikler, gamification, çoklu sınav sistemi, ileri predictive analytics ve kapsamlı teacher role sistemi backlog’da tutulur.

## 2. Temel Tasarım Prensipleri

**Personalization over Features:** Çok sayıda yüzeysel feature yerine öğrencinin gerçekten daha iyi çalışmasını sağlayan birkaç güçlü akış.

**Action over Analytics:** Grafik üretmek tek başına değer değildir; sistem “şimdi ne yapmalıyım?” ve “hangi öğrenciye müdahale etmeliyim?” sorularına cevap vermelidir.

**Data over Generic AI:** LLM genel eğitim tavsiyesi vermek yerine Luminary’nin structured student data’sını kullanılmalıdır.

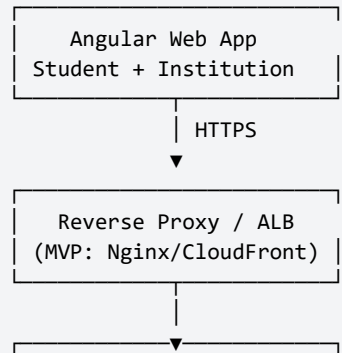
**Deterministic facts, AI interpretation:** Net, doğruluk, risk skoru ve trend gibi gerçekler backend/analytics tarafından hesaplanır. AI bu gerçekleri yorumlar ve doğal dile çevirir.

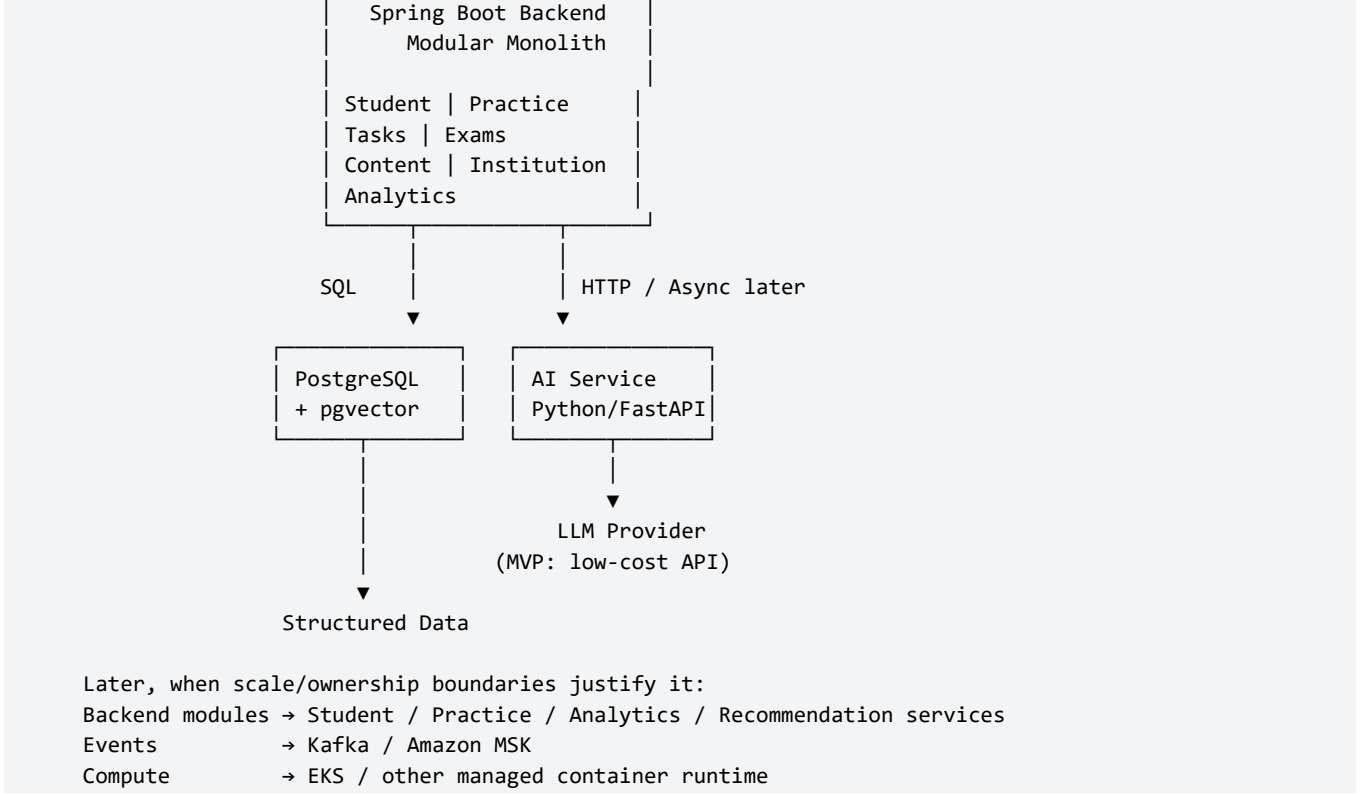
**MVP simplicity:** İki kişilik ekipte operasyonel yükü yüksek altyapı parçaları ancak gerçek ihtiyaç oluştuğunda devreye alınır.

**Evolutionary architecture:** İlk dağıtım basit olabilir; ancak domain sınırları, interface’ler ve event modelleri ileride microservice’e ayrıştırılabilecek şekilde tasarlanır.

## 3. Hedef Mimari: Modular Monolith → Selective Microservices

İlk bakışta Luminary için çok sayıda microservice kurmak cazip görünse de iki kişilik bir ekip açısından bu yaklaşım erken optimizasyon olur. Her servis kendi repository’si, Docker image’ı, deployment’ı, logları, health check’leri ve incident yüzeyi ile birlikte gelir. Bunun yerine business domain’lerini kod seviyesinde birbirinden ayıran bir modular monolith ile başlanır. AI servisi ise workload ve runtime gereksinimleri farklı olduğu için erken ayrıştırılabilecek tek doğal sınırdır.





Önemli nokta: bu diyagram hedef yönü gösterir. İlk MVP'nin aynısını üretmek için tüm kutuları gerçek ayrı sistemler olarak kurmak zorunda değiliz.

## 4. Neden Bu Bölümleme?

Luminary'de domain sınırları teknik sınırlar kadar önemlidir. Örneğin Practice domain'i sorunun üretilmesi, sunulması ve cevabının kaydedilmesinden sorumluyken Analytics domain'i "öğrenci burada tekrar tekrar zorlanıyor mu?" sorusunu cevaplar. Böylece aynı veriyi kullanan fakat farklı sorumluluklara sahip alanlar ayrıştırılmış olur.

| Domain      | Temel sorumluluk                              | MVP'de formu            |
|-------------|-----------------------------------------------|-------------------------|
| Student     | Profil, hedef, kullanıcıya ait temel bilgiler | Backend modülü          |
| Institution | Kurum, öğrenci üyeliği, tenant sınırı         | Backend modülü          |
| Practice    | Soru seti, cevap, sonuç                       | Backend modülü          |
| Task        | Görevler, tamamlanma                          | Backend modülü          |
| Exam        | Deneme ve sonuçları                           | Backend modülü          |
| Content     | MEB içerikleri, chunk/embedding metadata      | Backend modülü + worker |
| Analytics   | Weakness, trend, risk metrikleri              | Backend modülü          |
| AI          | Soru üretimi, açıklama, yorumlama             | Ayrı Python service     |

## 5. Multi-Tenancy ve Data Isolation

B2B2C modelinde Luminary birden fazla kurumun verisini aynı platformda barındıracaktır. Örneğin ABC Dershanesi ve XYZ Dershanesi aynı uygulamayı kullanabilir ancak ABC'nin yöneticisi XYZ'nin öğrencilerini görmemelidir. Bu nedenle multi-tenancy yalnızca “ileride SaaS olursa ekleriz” niteliğinde bir özellik değildir; B2B ürünün temel güvenlik constraint'idir.

Öneri: başlangıçta shared database + shared schema + tenant\_id yaklaşımı. Her business entity'de tenant\_id bulunur ve servis katmanı bu tenant context'i authentication token'dan çıkarır. İlk müşteri sayısı düşükken her tenant için ayrı database veya ayrı schema operasyonel yükü gereksiz artırır.

```
students
-----
id
tenant_id
user_id
name
target_rank
created_at

-- Her sorgu tenant context ile kısıtlanır
SELECT * FROM students
WHERE tenant_id = :tenantId
AND id = :studentId;
```

### 5.1 Isolation neden kritik?

- Confidentiality: Bir kurumun öğrenci performansı başka bir kuruma sızmamalıdır.
- Authorization correctness: “Admin rolünde” olmak, bütün öğrencileri görmek anlamına gelmez; admin + tenant context birlikte değerlendirilmelidir.
- Operational safety: Yanlış tenant filtresi eklenmemiş bir query, ciddi bir veri sızıntısına dönüşebilir.
- Future scaling: Şimdiden tenant\_id kullanmak daha sonra tenant başına ayrı schema/database gibi stratejilere geçişi kolaylaştırır.
- Analytics boundary: National benchmark gibi aggregate analizler için tenant kimliklerinin analitik katmanda gereksiz şekilde taşınmaması gerekir.

Pratik uygulama: Spring Security authentication context → TenantContext → service/repository katmanı. Ayrıca integration testlerde “Tenant A kullanıcısı Tenant B verisine erişemez” senaryosu zorunlu güvenlik testi haline getirilmelidir.

## 6. Tech Stack

| Alan         | Seçim                 | Neden / kullanım                                                                           |
|--------------|-----------------------|--------------------------------------------------------------------------------------------|
| Frontend     | Angular + TypeScript  | İki ayrı experience için güçlü UI platformu; mevcut ekip deneyimiyle uyumlu.               |
| Core backend | Java 21 + Spring Boot | Business logic, REST API, security ve enterprise SaaS ihtiyaçları için.                    |
| Build        | Gradle                | Java build/test/package otomasyonu.                                                        |
| API          | REST + OpenAPI        | Frontend ile basit, anlaşılır contract; ileride servis sınırlarında tekrar kullanılabilir. |

|                  |                                                          |                                                                                            |
|------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Auth             | Amazon Cognito (hedef) / basit OIDC çözümü (MVP)         | JWT tabanlı authentication; MVP'de maliyet ve operasyon bütçesine göre sadeleştirilebilir. |
| DB               | PostgreSQL                                               | Transactional data + analytics aggregate tabloları için ana sistem.                        |
| Vector           | pgvector                                                 | MEB içerik RAG'i için PostgreSQL içinde vector search; ayrı vector DB maliyeti yok.        |
| Object storage   | S3 (hedef) / local-minio geliştirme                      | PDF, import dosyaları ve diğer object'ler için.                                            |
| AI service       | Python + FastAPI                                         | LLM/RAG pipeline'larını Java business backend'inden ayırmak.                               |
| LLM              | Provider abstraction + düşük maliyetli model             | İlk etapta model sağlayıcısı değiştirilebilir tutulur.                                     |
| Cache            | Redis / ElastiCache (backlog)                            | Ancak ölçülen bottleneck oluşursa.                                                         |
| Events           | Domain events → Kafka/MSK (backlog)                      | MVP'de interface ile soyutlanmış synchronous/outbox yaklaşımı yeterli olabilir.            |
| Container        | Docker                                                   | Yerel geliştirme ve taşınabilir deployment.                                                |
| CI/CD            | GitHub + GitHub Actions                                  | Kod, PR, test, image build ve deployment pipeline merkezi.                                 |
| IaC              | Terraform (hedef)                                        | AWS altyapısını kodla yönetmek; MVP'de yalnız gerekli kaynaklar.                           |
| Deploy           | Basit container runtime → EKS (backlog)                  | Scale ihtiyacına göre yükseltilir.                                                         |
| Observability    | Structured logs + basic metrics → OTel/Grafana (backlog) | İlk aşamada basit; production büyüdükçe derinleştirilir.                                   |
| Testing          | JUnit, Mockito, Testcontainers                           | Backend unit/integration testleri; gerçek PostgreSQL davranışını test etmek.               |
| Frontend testing | Angular test framework                                   | Component/service testleri.                                                                |

## 7. GitHub Merkezli Geliştirme Modeli

Kaynak kodu GitHub üzerinde tutulur. İki kişilik ekipte GitHub yalnızca git remote değil; issue, pull request, code review, CI/CD ve release yönetiminin merkezi olmalıdır.

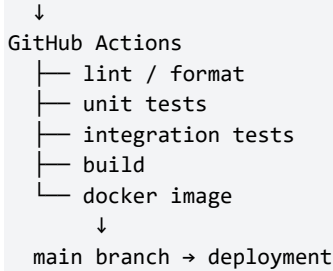
GitHub Repository

```

|
├─ frontend/
├─ backend/
├─ ai-service/
├─ infra/
└─ docs/

```

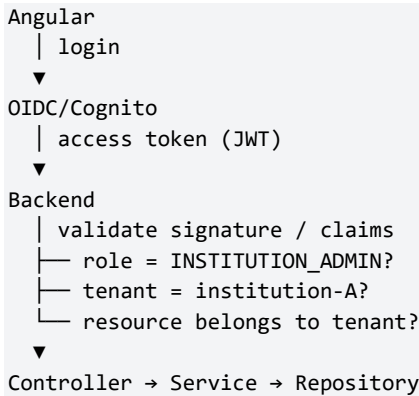
Pull Request



Branch yaklaşımı: iki kişi için GitFlow gibi ağır bir model yerine feature branch → Pull Request → kısa review → main tercih edilir. Her PR küçük ve tek sorumluluklu tutulur. “İki kişi olduğumuz için review gereksiz” yaklaşımı önerilmez; en az bir kişinin review ettiği PR’lar teknik borcu ciddi azaltır.

## 8. Authentication ve Authorization Flow

Önerilen üretim hedefinde Cognito/OIDC kullanıcıyı authenticate eder. Uygulama backend’i ise token’daki subject, role ve tenant bilgilerini request context’e taşır. Authorization iki boyutludur: rol + resource ownership/tenant.

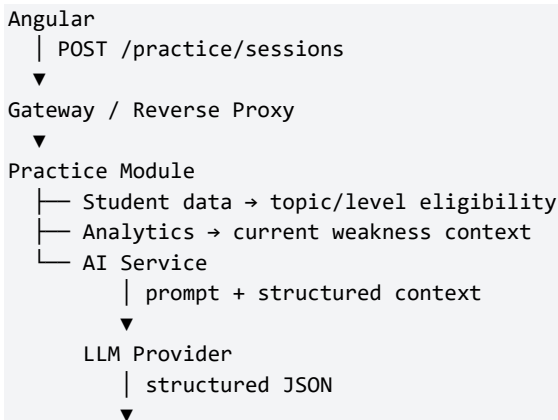


Örneğin Institution A admin’i /students/123 isteği yaptığında yalnızca öğrencinin tenant\_id’si A ise cevap döndürülmelidir. “Student ID’yi biliyorsa erişebilir” bir authorization modeli değildir.

## 9. Feature Bazlı Teknik Akışlar

### 9.1 AI Practice Lab

Öğrenci ders, konu, seviye ve soru sayısını seçer. Frontend doğrudan LLM’e gitmez. Backend önce authentication, rate limit, parametre doğrulama ve öğrencinin mevcut weakness context’ini kontrol eder.



```

Question schema validation
├─ malformed? → reject / retry
├─ duplicate/unsafe? → reject or regenerate
└─ valid → persist
      ▼
PostgreSQL
      ▼
Angular ← question set

```

Kritik ayrım: AI soru üretebilir fakat sorunun sistemde geçerli bir entity haline gelmesini backend belirler. Bu sayede LLM çıktısına körü körüne güvenilmez.

## 9.2 Question Solving → Weakness Data

```

Student answers question
      ↓
POST /practice/questions/{id}/answer
      ↓
Practice Module
      ↓
PostgreSQL (answer + result)
      ↓
Domain Event / Outbox record
      ↓
Analytics processor
      ↓
student_topic_metrics
      ↓
Weakness / trend / risk calculations

```

MVP’de Kafka olmadan bu olay bir application event + transaction sonrası worker tarafından işlenebilir. İleride Kafka eklendiğinde aynı event contract kullanılır.

## 9.3 Weakness Map

Weakness Map, AI’nın “fikri” değil, ölçülebilir öğrenci davranışlarının bir modelidir. Örneğin accuracy, recency, attempt count, difficulty ve exam performance gibi faktörler weighted rule set ile bir skor üretebilir.

```

Student 123
MATHEMATICS
  Problems      0.28 → WEAK
  Functions     0.52 → MEDIUM
  Polynomials   0.86 → STRONG

```

Analytics facts → Recommendation → optional AI explanation

Bu yaklaşım, aynı öğrencinin farklı günlerde neden farklı öneriler aldığını açıklayabilmek için de önemlidir. “Model böyle düşündü” yerine “son 3 denemede doğruluk %X düştü ve son 7 günde bu konuda yalnızca Y soru çözüldü” gibi izlenebilir gerekçeler üretilebilir.

## 9.4 My Tasks

```

GET /students/me/tasks
      ↓
Task Module
      ↓
PostgreSQL

Weakness / Risk event
      ↓
Recommendation rule

```



Task created

Example:

Problems accuracy < threshold

→ 15 Problems practice task

İlk aşamada task generation deterministic olmalıdır. AI, task'ın neden önerildiğini açıklayabilir; task'ın gerçek üretim mantığı analytics/recommendation katmanında kalır.

## 9.5 MEB Smart Study / RAG

MEB içerikleri object storage'da tutulur. Ingestion pipeline PDF → text → chunk → embedding → pgvector adımlarını izler. Öğrencinin sorusu geldiğinde tüm kitap LLM'e gönderilmez; en alakalı chunk'lar retrieval ile seçilir.

MEB PDF



Object Storage



Content ingestion worker



Text extraction



Chunking



Embedding model



PostgreSQL + pgvector

Student: "Bu sayfayı anlat."



Query embedding



Vector similarity search



Top-k relevant chunks



AI Service / LLM



Grounded explanation

Bu yapı hallucination riskini azaltır ve açıklamanın ilgili içerikten türetilmesini sağlar. Aynı zamanda ayrı bir vector DB satın alma/işletme ihtiyacını ilk aşamada ortadan kaldırır.

## 9.6 AI Study Coach

AI Coach ayrı bir "boş chatbot" değildir. Kullanıcının sorusuna cevap verirken structured context toplar: weakness map, son görevler, deneme trendleri, hedef ve son çalışma aktiviteleri. Sonra LLM bu context'i doğal dile çevirir.

Student: "Bugün ne çalışmalıyım?"

Student profile

Weakness map

Tasks

Exam trends

Goals

→ AI context → LLM → explanation

Recommendation Engine decides the activity.

AI explains the recommendation.

## 9.7 Institution Dashboard

Dashboard request'i her seferinde yüzlerce öğrencinin tüm raw activity kayıtlarını join edip baştan hesaplamamalıdır. Analytics layer önceden aggregate edilmiş student/institution metrics üretir.

```

Student activity
  ↓
Analytics calculation
  ↓
student_daily_metrics
student_topic_metrics
student_exam_trends
risk_scores
  ↓
Institution dashboard
  ↓
"Bu hafta incelenmesi önerilen 17 öğrenci"

```

Bu pattern başlangıçta da faydalıdır; Redis veya Kafka kullanmak zorunda olmadan PostgreSQL içinde summary tabloları oluşturulabilir.

## 9.8 Risk Detection

Risk Detection'ın MVP'de rule-based olması daha doğrudur. Örneğin aktivite düşüşü, deneme performans düşüşü ve zayıf konularda pratik eksikliği birleşerek risk puanı üretebilir.

```

risk_score = inactivity_weight
             + exam_decline_weight
             + weakness_persistence_weight
             + practice_gap_weight

0-29   HEALTHY
30-59  ATTENTION
60+    HIGH_RISK

```

Daha sonra bu kurallar gerçek pilot verileriyle kalibre edilebilir. Predictive ML, veri hacmi ve doğrulanabilir fayda oluşmadan önce backlog'da kalmalıdır.

## 9.9 Exam Management

İlk MVP'de kurumun deneme sonuçlarını CSV/Excel üzerinden içeri aktarması yeterlidir. Import pipeline veriyi validate eder, student mapping yapar ve transaction içinde kaydeder. Daha sonra exam result event'i analytics layer'ı tetikler.

```

Institution UI
  ↓
CSV / Excel upload
  ↓
Exam validation
  ├── student exists?
  ├── subject valid?
  ├── score / net valid?
  └── duplicate?
  ↓
PostgreSQL
  ↓
ExamCompleted / ResultImported event
  ↓
Analytics → trend + weakness + risk

```

## 10. Core Data Model

| Entity           | Örnek alanlar                                  | Not                        |
|------------------|------------------------------------------------|----------------------------|
| Institution      | id, name, status                               | Tenant root entity         |
| User             | id, auth_subject, role                         | Kimlik provider ile ilişki |
| Student          | id, tenant_id, user_id, target_rank            | Öğrenci domain root        |
| Subject          | id, exam_type, name                            | TYT bağlamı                |
| Topic            | id, subject_id, name                           | Weakness granularity       |
| Question         | id, topic_id, difficulty, source               | AI veya repository source  |
| QuestionAttempt  | id, student_id, question_id, correct, duration | Ham performans verisi      |
| Task             | id, student_id, type, target, status           | Öneri/çalışma aksiyonu     |
| Exam             | id, tenant_id, name, date                      | Deneme tanımı              |
| ExamResult       | id, exam_id, student_id, subject, net          | Longitudinal data          |
| WeaknessSnapshot | id, student_id, topic_id, score, created_at    | Zaman serisi               |
| StudentMetric    | student_id, period, metric_name, metric_value  | Dashboard aggregation      |

## 11. Event Model ve Outbox Yaklaşımı

Event-driven mimari uzun vadede faydalı olsa da ilk günden Kafka kurmak zorunda değiliz. Bunun yerine domain event contract'larını şimdiden tanımlayıp MVP'de transactional outbox veya basit post-commit processing kullanabiliriz.

```
QuestionSolvedEvent
```

```
{
  eventId,
  eventType,
  studentId,
  tenantId,
  questionId,
  topicId,
  correct,
  timestamp
}
```

```
ExamResultImportedEvent
```

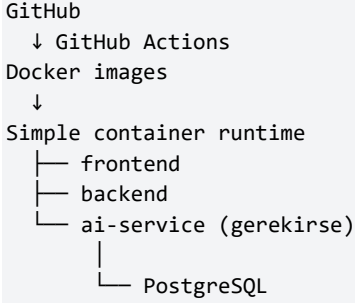
```
{
  eventId, tenantId, examId, studentId, ...
}
```

Transactional outbox yaklaşımı özellikle “DB’ye kayıt oldu ama event publish edilemedi” gibi iki sistem arasında tutarsızlık risklerini azaltır. Kafka geldiğinde outbox tablosu producer’ın kaynağı olabilir.

## 12. Deployment Strategy: Gerçekçi MVP

Hedef mimari AWS/EKS yönüne gidebilir; ancak iki kişilik ekip ve düşük sermaye nedeniyle başlangıçta EKS zorunlu tutulmamalıdır. İlk production/pilot deployment mümkün olduğunca az managed service ile kurulmalıdır.

MVP deployment



Files → object storage when required

Scale trigger oluşunca:

Simple runtime → EKS

Direct event processing → Kafka

DB cache pressure → Redis

PostgreSQL analytics → dedicated warehouse

Bu yaklaşım “AWS kullanmayalım” demek değildir. AWS, ihtiyaç duyulan yerde kullanılabilir; fakat her AWS servisini ilk günden satın almak yerine minimum operational surface hedeflenir.

## 13. Observability

İki kişilik ekipte observability “devasa monitoring platformu” anlamına gelmemelidir. İlk sürümde tüm backend logları structured JSON olabilir; request ID/correlation ID ile backend ve AI çağrıları izlenebilir. Error rate, latency, database health ve LLM failure gibi birkaç kritik sinyal yeterlidir.

Daha sonra OpenTelemetry + Prometheus + Grafana + Loki gibi daha kapsamlı bir stack kurulabilir. Buradaki prensip önce görünülük, sonra platform olgunluğudur.

## 14. Security ve Privacy

- JWT signature ve issuer/audience validation yapılmalı.
- Tenant/resource authorization her protected endpoint’te uygulanmalı.
- Parolalar uygulama DB’sinde tutulmamalı; OIDC/Cognito gibi identity provider kullanılmalı.
- AI provider’a gereksiz kişisel veri gönderilmemeli; student ID yerine gerektiğinde internal pseudonymous identifiers kullanılmalı.
- MEB içerikleri ile öğrenci özel verisi aynı security domain içinde düşünülmemeli.
- Secret’lar GitHub repository’ye commit edilmemeli; GitHub Actions secrets / environment secrets kullanılmalı.
- Database backup ve restore süreci MVP’den önce en az bir kez test edilmeli.

## 15. Testing Strategy

| Katman | Test türü | Örnek |
|--------|-----------|-------|
|--------|-----------|-------|

|                |                               |                                                     |
|----------------|-------------------------------|-----------------------------------------------------|
| Backend domain | Unit                          | Weakness scoring, risk scoring, task recommendation |
| REST API       | Integration                   | Authorization, validation, tenant isolation         |
| Database       | Testcontainers                | PostgreSQL query/migration behavior                 |
| AI integration | Contract / mocked integration | LLM JSON schema, retries, timeout                   |
| Frontend       | Component/service             | Practice flow, dashboard rendering                  |
| E2E            | Selected critical paths       | Login → practice → answer → metrics                 |
| Security       | Negative tests                | Tenant A cannot access Tenant B                     |

AI testleri yalnız “cevap doğru mu?” şeklinde ele alınmamalı. JSON schema validity, timeout, provider failure, empty retrieval result ve prompt injection gibi failure modes da test edilmelidir.

## 16. Roadmap

Roadmap, ürün raporundaki Foundation → Institution MVP → Intelligence → Scale sıralamasını teknik delivery planına dönüştürür.

| Sprint | Hedef                 | Çıktılar                                                                                                       |
|--------|-----------------------|----------------------------------------------------------------------------------------------------------------|
| 1      | Foundation            | GitHub repo, branching, CI, Docker, Spring Boot skeleton, Angular skeleton, PostgreSQL, migrations, auth spike |
| 2      | Student Core          | Student profile, subjects/topics, tenant model, authorization baseline                                         |
| 3      | Practice Lab          | Practice session API, question model, AI generation spike, validation, answer submission                       |
| 4      | Performance           | QuestionAttempt, basic metrics, domain events/outbox, first weakness scoring                                   |
| 5      | Weakness + Tasks      | Weakness map UI/API, rule-based task recommendation, task lifecycle                                            |
| 6      | MEB Smart Study       | PDF ingestion, chunking, embeddings, pgvector retrieval, grounded explanation                                  |
| 7      | Institution Core      | Institution, student management, admin dashboard shell                                                         |
| 8      | Exams                 | Exam creation/import, result mapping, validation, trend data                                                   |
| 9      | Institution Analytics | Performance dashboard, subject weakness, student profile                                                       |
| 10     | Risk                  | Risk rules, alerts, intervention-oriented dashboard                                                            |

|    |                 |                                                                                 |
|----|-----------------|---------------------------------------------------------------------------------|
| 11 | Pilot Hardening | Security, tenant isolation tests, observability, backup/restore, error handling |
| 12 | Pilot Release   | Deployment, onboarding 100–300 student pilot, KPI instrumentation               |

## 17. Post-MVP Intelligence Roadmap

- Recommendation Engine v2
- AI Study Coach with structured student context
- Historical exam trend analysis
- Goal-based improvement gap
- Personalized study plan
- Institution benchmarking
- Teacher / Guidance Counselor roles
- Multi-branch management
- National weakness map
- University / department targeting
- Parent summaries
- Additional exams

Bu maddelerden hiçbirisi ilk MVP'nin ön koşulu değildir. Özellikle predictive ML, national benchmarking ve university targeting ancak yeterli longitudinal data oluştuğunda anlamlı hale gelir.

## 18. 2 Kişilik Ekip ile Çalışma Modeli

İki kişi için ownership alanlarının kaba şekilde paylaşılması verimlidir; ancak iki ayrı ürün ekibi kurmak yerine feature'ın uçtan uca teslim edilmesi hedeflenmelidir.

| Kişi        | Birincil odak          | İkincil sorumluluk                                              |
|-------------|------------------------|-----------------------------------------------------------------|
| Developer A | Backend / architecture | Database, security, CI/CD, integrations                         |
| Developer B | Frontend / product     | UX flows, frontend tests, API integration, analytics views      |
| Ortak       | AI/product logic       | Prompting, RAG quality, recommendation rules, product decisions |

Haftalık çalışma biçimi: kısa planning → küçük GitHub issues → feature branch → PR → review → merge → otomatik CI. Büyük “iki haftalık dev feature” PR'larından kaçınılır.

## 19. Backlog: Başlangıçta Kurulmaması Gereken Altyapılar

| Teknoloji / yaklaşım | Neden ertelenebilir?                                    | Ne zaman devreye alınır?                                                     | İlk alternatif                  |
|----------------------|---------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------|
| Kubernetes / EKS     | İki kişilik ekip için deployment/operasyon yükü yüksek. | Birden fazla servis, autoscaling, deployment karmaşıklığı anlamlı olduğunda. | Tek/az sayıda container runtime |

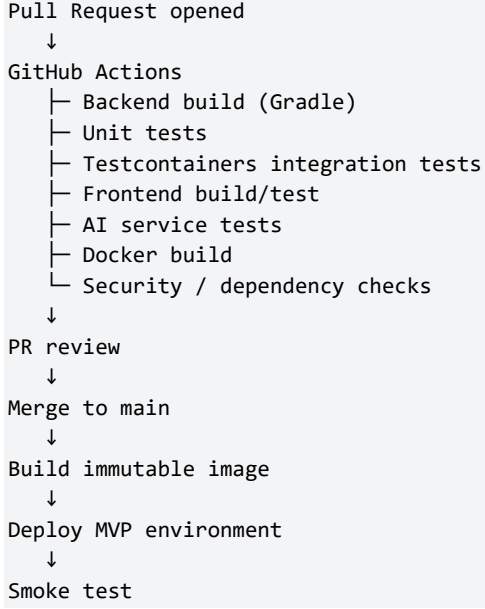
|                               |                                                            |                                                                         |                                                   |
|-------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------|
| Kafka / MSK                   | MVP trafik ve event hacmi için gereksiz operasyonel yüzey. | Async processing, event replay, bağımsız consumer sayısı arttığında.    | Application events + outbox + worker              |
| Redis / ElastiCache           | Başta DB sorguları ölçeği kaldırabilir.                    | Ölçülen latency/cache hit ihtiyacı oluştuğunda.                         | PostgreSQL + uygun index + küçük in-process cache |
| Dedicated analytics warehouse | Pilot hacimde gereksiz.                                    | Analitik sorgular operational DB'yi etkilemeye başladığında.            | PostgreSQL aggregate tables                       |
| ArgoCD                        | İkinci deployment katmanı operasyonel yük getirir.         | Birden fazla env / cluster ve sık GitOps deployment ihtiyacı olduğunda. | GitHub Actions deployment                         |
| Full OpenTelemetry stack      | Kurulum/işletim maliyeti erken aşamada gereksiz.           | Distributed debugging gerçekten gerekli olduğunda.                      | Structured logs + correlation IDs + basic metrics |
| Microservice per domain       | Network, deployment ve debugging yükü oluşturur.           | Domain ownership ve scale bağımsızlığı gerçek bottleneck olduğunda.     | Modular monolith                                  |
| Predictive ML                 | Yeterli longitudinal veri olmayabilir.                     | Pilot data ile baseline ve label kalitesi oluştuğunda.                  | Rule-based risk/recommendation                    |
| Separate vector DB            | Ek maliyet ve operasyon.                                   | Vector ölçeği PostgreSQL'in pratik sınırlarını aştığında.               | pgvector                                          |
| Multi-region                  | MVP için gereksiz.                                         | Gerçek SLA/availability gereksinimi oluştuğunda.                        | Single region + backups                           |

## 20. Refactor Tetikleyicileri: “Ne zaman yükseltmeliyiz?”

Her teknolojinin backlog'a alınması “zamanı geldiğinde hissederiz” şeklinde olmamalıdır. Ölçülebilir trigger'lar kullanılmalıdır.

| Trigger                                        | Mevcut çözüm                | Upgrade                             |
|------------------------------------------------|-----------------------------|-------------------------------------|
| API latency DB yüzünden yükseliyor             | Index + query optimization  | Redis / read replica                |
| Async job backlog büyüyor                      | Single worker               | Queue/Kafka + multiple workers      |
| Deployment süresi/riski büyüyor                | Single container deployment | EKS + GitOps                        |
| AI service CPU/memory bağımsız scale gerekiyor | Same host/container         | Dedicated AI compute/service        |
| Dashboard queries operational DB'yi etkiliyor  | Aggregate tables            | Dedicated analytics store/warehouse |
| Multiple teams/owners emerge                   | Modular monolith            | Selective microservices             |
| Observability gap across services              | Correlation ID + logs       | OpenTelemetry + tracing             |

## 21. GitHub + CI/CD Detayı

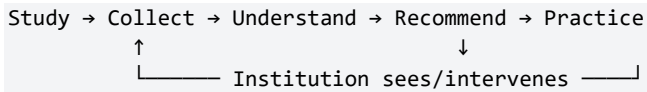


Branch protection ile main'e doğrudan push kapatılabilir. İki kişi olduğumuz için süreç basit tutulmalı; ancak testlerin bypass edilememesi ve PR review zorunluluğu korunmalıdır.

## 22. Örnek Uçtan Uca Senaryo

Senaryo: Bir öğrenci son üç çalışmada TYT Matematik / Problemler konusunda düşük doğruluk göstermiş olsun. Aynı anda son deneme sonucu da düşmüş olsun. Luminary bu durumu nasıl aksiyona dönüştürür?

1. Öğrenci problem soruları çözer; QuestionAttempt kayıtları oluşur.
2. Analytics bu sonuçlardan topic accuracy ve recency metriklerini günceller.
3. Weakness scorer "Problems = WEAK" sonucunu üretir.
4. Risk engine, performans düşüşünü ve practice gap'i risk skoruna ekler.
5. Recommendation engine "15 problems practice" önerir ve task üretir.
6. Öğrenci Home ekranında task'ı görür.
7. Öğrenci isterse "neden bunu yapmalıyım?" diye AI Coach'a sorar.
8. AI Coach structured metrics'i okuyup açıklama üretir.
9. Kurum admin'i dashboard'da aynı öğrenciyi "Attention Required" olarak görür.
10. Yeni practice/exam data geldikçe cycle tekrar eder.



## 23. MVP Başarı Kriterleri

| Alan               | Ölçüm                                              |
|--------------------|----------------------------------------------------|
| Student activation | İlk kayıt sonrası ilk practice/task kullanım oranı |
| Weekly engagement  | WAU/MAU, weekly study sessions                     |

|              |                                                     |
|--------------|-----------------------------------------------------|
| Practice     | Sorular çözüldü, accuracy, repeat usage             |
| Tasks        | Tamamlama oranı                                     |
| Exam         | Sonuçların sisteme girilme oranı                    |
| Institution  | Admin weekly active usage                           |
| Intervention | Risk alert sonrası kurum aksiyonu                   |
| Outcome      | Çalışma ve performans trendlerinde anlamlı iyileşme |

Bu metrikler ürünün gerçek değerini test etmek için kullanılmalı; “kaç feature yaptık?” bir başarı metriği değildir.

## 24. İlk Üretim İçin Önerilen Minimum Altyapı

### REQUIRED NOW

- └─ GitHub
- └─ GitHub Actions
- └─ Angular
- └─ Spring Boot / Java 21
- └─ PostgreSQL
- └─ Docker
- └─ Basic object storage
- └─ Basic auth / OIDC
- └─ AI provider abstraction

### BACKLOG / NEED-DRIVEN

- └─ EKS
- └─ Kafka / MSK
- └─ Redis / ElastiCache
- └─ ArgoCD
- └─ Dedicated analytics warehouse
- └─ Full OTel stack
- └─ Multi-region
- └─ Predictive ML

Bu liste maliyet tasarrufu kadar ekip odağı için de önemlidir. İki kişinin zamanının önemli bölümünün cluster, ingress, broker, cache ve observability operasyonuna gitmesi, ürünün asıl value proposition’ı olan personalization loop’un gecikmesine yol açabilir.

## 25. Notlar ve Maliyet Constraint’ine Göre Refactor Kararları

Bu dokümanın tasarım ve tech stack bölümleri, Luminary’nin orta/uzun vadede büyüyebileceği hedef mimariyi korur. Ancak mevcut başlangıç koşulu iki kişi ve sınırlı sermayedir. Bu nedenle aşağıdaki yaklaşım özellikle benimsenmelidir: hedef mimari ile ilk deployment aynı şey değildir.

İlk aşamada ücretsiz/açık kaynak ve düşük operasyonel maliyetli çözümler tercih edilmelidir. GitHub + GitHub Actions, Docker, Spring Boot, Angular, PostgreSQL, Testcontainers ve pgvector bunun doğal temelidir. Geliştirme ortamında Docker Compose kullanılabilir; ekip lokalinde ihtiyaç duyulan Postgres ve yardımcı servisler container olarak kaldırılabilir.

AWS servisleri ihtiyaç bazlı açılmalıdır. Özellikle EKS, MSK, ElastiCache ve dedicated analytics sistemleri backlog’da tutulabilir. Tek bir PostgreSQL instance’ı ile başlanıp index’ler ve aggregate tablolar doğru tasarlanırsa pilot hacminin

önemli bölümü burada çözülebilir. Aynı şekilde Kafka yerine application event + outbox + background worker ile event-driven davranışın büyük kısmı elde edilebilir.

Microservice hedefi korunur; fakat ilk kod tabanı modular monolith olur. Modüller arası bağımlılıklar interface/DTO/event contract'ları üzerinden tanımlanır. Bir modül gerçek bir bottleneck olduğunda, örneğin Analytics sürekli compute tüketmeye başladığında veya AI workload'u backend'den bağımsız scale olmak istediğinde, yalnızca o sınır microservice'e çıkarılır. Böylece "big-bang rewrite" ihtiyacı azaltılır.

ArgoCD de ilk günden zorunlu değildir. GitHub Actions'ın build-test-deploy akışı yeterliyse kullanılabilir. Aynı şekilde Kubernetes "best practice olduğu için" değil, gerçekten birden fazla workload, rollout, autoscaling veya cluster yönetimi ihtiyacı oluştuğunda alınmalıdır.

En önemli prensip: ucuz başlamak, kötü tasarlamak anlamına gelmemelidir. Veri modeli, tenant isolation, authorization, API contract'ları, event schema'ları ve domain sınırları baştan doğru kurulmalıdır. Buna karşılık deployment platformu, message broker, cache ve analytics infrastructure gibi uygulamanın etrafındaki operasyonel bileşenler ihtiyaç çıktıkça refactor edilebilir.

Sonuç olarak başlangıç için önerilen yol: küçük ekip + küçük maliyet + güçlü domain model + güçlü data pipeline + ölçülebilir MVP. Luminary'nin rekabet avantajı infrastructure sayısı değil; Student Data → Understanding → Recommendation → Action döngüsünün gerçekten çalışmasıdır.

## 26. Sonuç

Luminary için teknik hedef, öğrencinin ve kurumun aynı data ecosystem'ini kullandığı, AI'nın structured data üzerine oturduğu, tenant-aware bir B2B2C platform oluşturmaktır. İlk MVP modular monolith + PostgreSQL + Angular + Java/Spring Boot + GitHub/GitHub Actions + düşük maliyetli AI provider yaklaşımıyla geliştirilebilir. Raporun geri kalanında tanımlanan Kafka, EKS, Redis, dedicated analytics, advanced observability ve predictive ML gibi bileşenler mimarinin yönünü gösterir; fakat iki kişilik ekip ve sınırlı sermaye nedeniyle ancak ölçülmüş ihtiyaç oluştuğunda devreye alınmalıdır.

Bu strateji hem kısa vadede hızlı pilot çıkarmayı hem de ürün-market fit görüldüğünde mimariyi kontrollü şekilde büyütmeyi sağlar.